

1. Install fastlane by using `gem install fastlane` or `brew cask install fastlane`.
2. Make your own Flutter project, and once prepared, ensure that your project builds via
 - `flutter build apk --release`; and
 - `flutter build ios --release --no-codesign`.
3. Initializing the fastlane comes for every platform.
 - In your `[project]/android` directory, run `fastlane init`.
 - In your `[project]/ios` directory, run `fastlane init`.
4. Edit the Appfiles to make sure they have adequate information for your app.
 - Ensure that the `package_name` inside `[project]/android/Appfile` is same as your package name in `AndroidManifest.xml`.
 - make certain `app_identifier` in `[project]/ios/Appfile` jointly matches `info.plist`'s bundle symbol. Fill the details like `apple_id`, `itc_team_id`, `team_id` together with your individual account information.
5. Come up with your native login credentials for the stores.
 - Follow offer, the availability and the provision setup steps and make sure that fastlane supply in syncs information successfully from your Play Store console. You need to treat the `.json` file like your word file and don't check it into any public supply management repositories.
 - Your iTunes Connect username is already present in your Appfile's `apple_id` field, so you just need to set the `FASTLANE_PASSWORD` shell setting variable together with your iTunes Connect word. Otherwise, you'll be prompted once uploading to iTunes/TestFlight.
6. Come up with a code sign language.
 - On Android, there are two signing keys that are present namely, deployment and upload. The end-users transfer the `.apk` signed along with the 'deployment key' and 'upload key' and it is employed to evidence the `.apk` uploaded by developers onto the Play Store which is then re-signed with the deployment key once within the Play Store.
 - It's highly recommended to use the automated cloud managed signing for the deployment key. For a lot of data, see the official Play Store documentation.

Follow the given steps for key generation to create an upload key:

- Configure gradle to use your upload key once building your app